

WEB INFRASTRUCTURE SUMMATIVE GROUP PRACTICAL REPORT

Securing and Scaling a Production-Style Web Infrastructure

Field	Details
Module	Web Infrastructure (BSc(Hons) Software Engineering, Year 1)
Faculty / Facilitator	POHUR Navindranath Sharma
Submission date	September 2026
Environment used	Ubuntu 24.04 LTS (64-bit), Single Virtual Machine on Oracle VM VirtualBox 7.1, 2 vCPUs, 4GB RAM, Host: Windows 11, VM IP: 192.168.56.101 (Host-Only) and 127.0.0.1 (Loopback)
Group Members	Nelson Fodjo Kamdoun Rebecca Adhiambo Oduor Leatitia Mizero Edith Otieno
GitHub Url	https://github.com/Becca4375/group_03_secure-web-infra.git
Technical Video Url	https://drive.google.com/file/d/1lnefyBB-NAYc0mPnDRjgVhUBvwNcnKQU/view?usp=sharing

Table of Contents

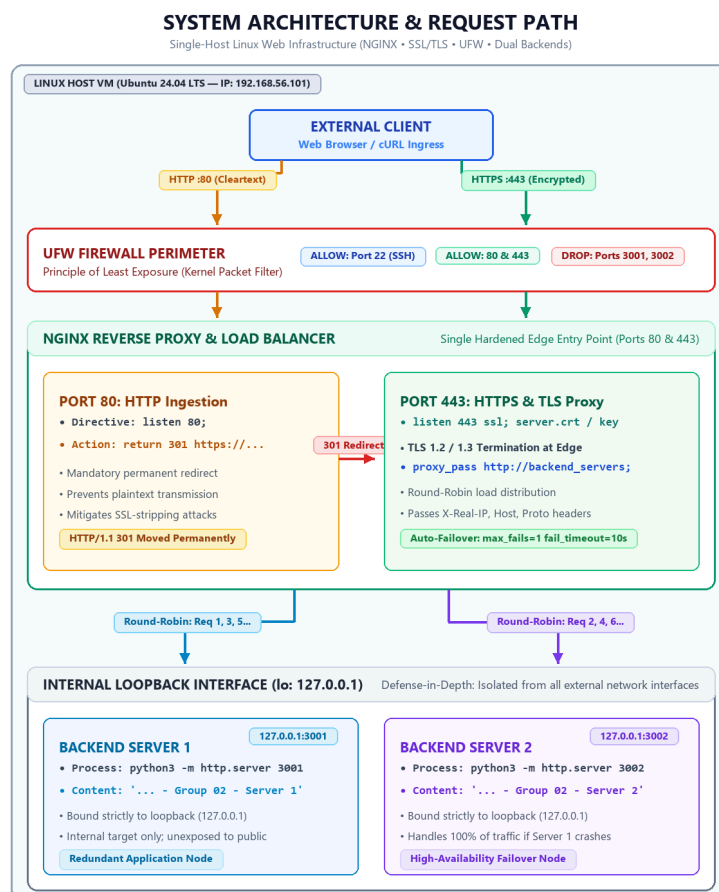
Table of Contents.....	2
1. Executive Summary & Overview.....	3
2. Infrastructure Architecture & Request Path.....	3
2.1 Infrastructure diagram.....	3
2.2 Step-by-Step Request Path Explanation.....	4
3. Backend Environment Setup.....	5
3.1 Implementation Details.....	5
3.2 Evidence and Verification.....	9
4. NGINX Reverse Proxy and Load Balancing.....	10
4.1 Foundations: Reverse Proxying & Load Balancing.....	10
4.2 Configuration Details.....	11
4.3 Validation & Load Balancing Evidence.....	12
4.4 Backend Resilience & Single-Instance Failure Test.....	13
5. SSL/TLS & HTTPS Configuration.....	15
5.1 Foundations: Cryptography, TLS, and Redirection.....	15
5.2 Certificate Generation & Security Roles.....	16
5.3 Certificate Inspection.....	17
5.4 NGINX HTTPS & Port 80 Redirect Configuration.....	17
5.5 HTTPS Verification Evidence.....	18
6. Firewall Configuration and Verification (UFW).....	19
6.1 Principle of Least Exposure & Port Strategy.....	19
6.2 Pre-Configuration Baseline & Safety Measures.....	20
6.3 Firewall Configuration Commands.....	20
6.4 Firewall Evidence & Connection Testing.....	21
6.5 Defense-in-Depth: Socket Binding vs. Firewall Filtering.....	23
7. Troubleshooting & Diagnostic Reasoning.....	23
Stage 1: Problem and Symptom.....	23
Stage 2: Evidence Collection.....	23
Stage 3: Systematic Investigation & Hypothesis Testing.....	24
Stage 4: Root Cause Identification.....	25
Stage 5: Fix and Verification.....	25
8. Architecture & Security Theory Q&A.....	26
9. Conclusion.....	27
10. Sources, Credits & AI Transparency.....	27
AI Transparency DisclosureTable of Contents 2.....	28

1. Executive Summary & Overview

Modern web applications require three fundamental infrastructure pillars to be viable for production: high availability, encrypted transport security, and controlled network exposure. In this practical assignment, our group designed, deployed, hardened, and verified a production-style single-host Linux web infrastructure. Our implementation uses NGINX as an edge reverse proxy and layer-7 load balancer positioned in front of two independent local backend instances (Backend 1 listening on port 3001 and Backend 2 on port 3002). To enforce secure transport, we generated an X.509 SSL/TLS certificate pair with OpenSSL, enabled HTTPS on port 443, and set up a mandatory HTTP 301 redirect on port 80. We hardened network perimeters with UFW (Uncomplicated Firewall), following the Principle of Least Exposure, allowing only essential administration and web traffic while keeping backend services isolated to the loopback interface. Finally, we simulated and diagnosed an upstream service failure using a structured 5-stage diagnostic methodology backed by service inspection and log audits

2. Infrastructure Architecture & Request Path

2.1 Infrastructure diagram



2.2 Step-by-Step Request Path Explanation

- **1. Client Ingress & Firewall Inspection:** When an external client sends a request, the packet first hits the Linux kernel packet filter managed by UFW. If the packet targets port 80 or 443, it is allowed through; all connection attempts to unapproved ports (e.g., 3001 or 3002) are dropped.
- **2. HTTP Redirection (Port 80):** If the client sends an unencrypted HTTP request on port 80, NGINX receives it and immediately returns an HTTP/1.1 301 Moved Permanently response with the header Location: `https://$host$request_uri`, directing the client to the secure endpoint.
- **3. Encrypted HTTPS Handshake (Port 443):** The client reconnects on port 443. NGINX presents its public TLS certificate (`server.crt`), negotiates cipher suites with the client, and establishes an encrypted TLS session. NGINX terminates TLS at this edge boundary.
- **4. Reverse Proxying & Load Balancing:** NGINX decrypts the HTTP payload and uses a round-robin algorithm to forward the request to the upstream pool (127.0.0.1:3001 or 127.0.0.1:3002). NGINX adds proxy headers (`X-Real-IP`, `X-Forwarded-For`, and `X-Forwarded-Proto`) so the backends receive original client details.
- **5. Backend Processing & Delivery:** The selected Python backend handles the request over the loopback interface (`lo`) and returns its distinct HTML response to NGINX over the local socket.
- **6. Encrypted Egress Delivery:** NGINX re-encrypts the response using the negotiated TLS session keys and delivers it back to the client over port 443 with an HTTP/1.1 200 OK status.

3. Backend Environment Setup

3.1 Implementation Details

Two separate directories were established on the Linux machine: `~/backend1` and `~/backend2`. Each directory contains an `index.html` file with distinct, identifiable text to verify load distribution.

Baseline Checks: Initial system verification:

```
cat /etc/os-release
hostname
whoami
sudo systemctl status nginx
nginx -v
openssl version
sudo ufw status
```

```

becca@DESKTOP-DOK0N11: $ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
becca@DESKTOP-DOK0N11: $ hostname
DESKTOP-DOK0N11
becca@DESKTOP-DOK0N11: $ whoami
becca
becca@DESKTOP-DOK0N11: $ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-09-12 11:01:41 +04; 7min ago
 Invocation: 0a95441b2e534f708358bf14b73605bf
    Docs: man:nginx(8)
   Main PID: 234 (nginx)
      Tasks: 13 (limit: 9328)
  Memory: 11.7M (peak: 14M)
     CPU: 288ms
    CGroup: /system.slice/nginx.service
            └─234 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
              └─237 "nginx: worker process"
                └─238 "nginx: worker process"
                  └─239 "nginx: worker process"
                    └─240 "nginx: worker process"
                      └─241 "nginx: worker process"
                        └─242 "nginx: worker process"
                          └─243 "nginx: worker process"
                            └─245 "nginx: worker process"
                              └─246 "nginx: worker process"
                                └─247 "nginx: worker process"
                                  └─249 "nginx: worker process"
                                    └─250 "nginx: worker process"

Sep 12 11:01:40 DESKTOP-DOK0N11 systemd[1]: Starting nginx.service - A high performance web>
Sep 12 11:01:41 DESKTOP-DOK0N11 systemd[1]: Started nginx.service - A high performance web >

becca@DESKTOP-DOK0N11: $ nginx -v
nginx version: nginx/1.28.3 (Ubuntu)
becca@DESKTOP-DOK0N11: $ openssl version
OpenSSL 3.5.5 27 Jan 2026 (Library: OpenSSL 3.5.5 27 Jan 2026)
becca@DESKTOP-DOK0N11: $ sudo ufw status
Status: inactive

```

What this proves: the baseline checks confirm NGINX is active, OpenSSL is installed, and UFW starts inactive — establishing the pre-configuration state.

Commands executed to create directories and distinctive files:

```

mkdir -p ~/backend1 ~/backend2
echo "<h1>Web Infrastructure - Group 02 - Server 1</h1>" > ~/backend1/index.html
echo "<h1>Web Infrastructure - Group 02 - Server 2</h1>" > ~/backend2/index.html

```

Commands to start backend services on local loopback:

```
cd ~/backend1 && python3 -m http.server 3001 --bind 127.0.0.1 &  
cd ~/backend2 && python3 -m http.server 3002 --bind 127.0.0.1 &
```

```
becca@DESKTOP-DOK0N11:~$ cd ~/backend1  
becca@DESKTOP-DOK0N11:~/backend1$ python3 -m http.server 3001 --bind 127.0.0.1  
Serving HTTP on 127.0.0.1 port 3001 (http://127.0.0.1:3001/) ...  
█
```

```

becca@DESKTOP-DOK0N11:~$ cd ~/backend2
becca@DESKTOP-DOK0N11:~/backend2$ python3 -m http.server 3002 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 3002 (http://127.0.0.1:3002/) ...
^C
Keyboard interrupt received, exiting.
becca@DESKTOP-DOK0N11:~/backend2$ python3 -m http.server 3002 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 3002 (http://127.0.0.1:3002/) ...
127.0.0.1 - - [12/Sep/2026 11:41:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Sep/2026 11:41:50] code 404, message File not found
127.0.0.1 - - [12/Sep/2026 11:41:50] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [12/Sep/2026 11:50:59] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Sep/2026 12:00:44] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Sep/2026 12:35:02] code 404, message File not found
127.0.0.1 - - [12/Sep/2026 12:35:02] "GET /favicon.ico HTTP/1.0" 404 -
127.0.0.1 - - [12/Sep/2026 12:36:17] code 404, message File not found
127.0.0.1 - - [12/Sep/2026 12:36:17] "GET /favicon.ico HTTP/1.0" 404 -
127.0.0.1 - - [12/Sep/2026 12:36:17] code 404, message File not found
127.0.0.1 - - [12/Sep/2026 12:36:17] "GET /favicon.ico HTTP/1.0" 404 -
127.0.0.1 - - [12/Sep/2026 12:36:24] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:36:48] "GET / HTTP/1.0" 304 -
127.0.0.1 - - [12/Sep/2026 12:36:49] "GET / HTTP/1.0" 304 -
127.0.0.1 - - [12/Sep/2026 12:36:50] "GET / HTTP/1.0" 304 -
127.0.0.1 - - [12/Sep/2026 12:36:51] "GET / HTTP/1.0" 304 -
127.0.0.1 - - [12/Sep/2026 12:36:51] "GET / HTTP/1.0" 304 -
127.0.0.1 - - [12/Sep/2026 12:36:52] "GET / HTTP/1.0" 304 -
127.0.0.1 - - [12/Sep/2026 12:36:53] "GET / HTTP/1.0" 304 -
127.0.0.1 - - [12/Sep/2026 12:36:57] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:36:58] "GET / HTTP/1.0" 304 -
127.0.0.1 - - [12/Sep/2026 12:36:59] code 404, message File not found
127.0.0.1 - - [12/Sep/2026 12:36:59] "GET /favicon.ico HTTP/1.0" 404 -
127.0.0.1 - - [12/Sep/2026 12:37:55] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:05] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:07] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:16] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:17] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:18] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:19] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:20] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:20] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:21] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:22] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:23] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:24] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:24] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:25] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:26] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:29] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 12:41:30] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [14/Sep/2026 00:34:08] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [14/Sep/2026 00:34:08] code 404, message File not found
127.0.0.1 - - [14/Sep/2026 00:34:08] "GET /favicon.ico HTTP/1.0" 404 -

```

Why Distinguishable Content is Mandatory: If both backend services returned identical HTML, there would be no way to empirically prove that load balancing is functioning or to identify which server handled a given request. Having distinct text (Server 1 vs Server 2) provides definitive visual and programmatic proof of request distribution.

3.2 Evidence and Verification

```
becca@DESKTOP-DOK0N11:~/backend1$ curl http://127.0.0.1:3001
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 1</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 1</h1>
  </body>
</html>
becca@DESKTOP-DOK0N11:~/backend1$ curl http://127.0.0.1:3002
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 2</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 2</h1>
  </body>
</html>
becca@DESKTOP-DOK0N11:~/backend1$
```

Figure 3.1 — Verification of distinct local responses from Backend 1 (port 3001) and Backend 2 (port 3002) via curl.

```
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ ss -tulpn
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:*
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:*
udp UNCONN 0 0 10.255.255.254:53 0.0.0.0:*
udp UNCONN 0 0 127.0.0.1:323 0.0.0.0:*
udp UNCONN 0 0 127.0.0.1:323 0.0.0.0:*
udp UNCONN 0 0 [::1]:323 [::]:*
udp UNCONN 0 0 [::1]:323 [::]:*
tcp LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:*
tcp LISTEN 0 4096 127.0.0.54:53 0.0.0.0:*
tcp LISTEN 0 511 0.0.0.0:80 0.0.0.0:*
tcp LISTEN 0 4096 0.0.0.0:22 0.0.0.0:*
tcp LISTEN 0 511 0.0.0.0:443 0.0.0.0:*
tcp LISTEN 0 5 127.0.0.1:3002 0.0.0.0:* users:(("python3",pid=32621,fd=3))
tcp LISTEN 0 5 127.0.0.1:3001 0.0.0.0:* users:(("python3",pid=32615,fd=3))
tcp LISTEN 0 1000 10.255.255.254:53 0.0.0.0:*
tcp LISTEN 0 4096 [::]:22 [::]:*
```

Figure 3.2 — Socket inspection (ss -tulpn) proving Python services are bound exclusively to 127.0.0.1 loopback.

Evidence Explanation: Figure 3.1 demonstrates that running `curl http://127.0.0.1:3001` returns '`<h1>Web Infrastructure - Group 02 - Server 1</h1>`', while `curl http://127.0.0.1:3002` returns '`<h1>Web Infrastructure - Group 02 - Server 2</h1>`'. In Figure 3.2, running `ss -tulpn` shows two active TCP sockets bound strictly to 127.0.0.1:3001 and 127.0.0.1:3002 under Python process IDs, confirming both services are running and isolated from external interfaces.

4. NGINX Reverse Proxy and Load Balancing

4.1 Foundations: Reverse Proxying & Load Balancing

The Purpose of a Reverse Proxy: A reverse proxy is an intermediary server that sits in front of web applications and intercepts incoming client requests. While a forward proxy acts on behalf of clients to access external networks, a reverse proxy acts on behalf of backend servers. In our architecture, NGINX provides:

- **Topology Hiding:** Clients only ever see NGINX on port 80/443. The internal ports (3001, 3002) and backend technology stack remain completely hidden.
- **Centralized SSL/TLS Termination:** Cryptographic processing is offloaded to NGINX at the edge, freeing application runtimes from managing SSL/TLS certificates.
- **Request Filtering & Buffering:** Mitigates slow-client attacks and buffers requests before forwarding them to backend servers.

The Purpose of Load Balancing: Load balancing is the distribution of client requests across multiple backend instances to optimize throughput, reduce latency, and ensure high availability. If one backend instance fails, the load balancer reroutes traffic to the surviving healthy instance, eliminating a single point of failure (SPOF).

Operational Area	Direct Backend Access (http://host:3001)	Reverse Proxy Access (https://host/)
Network Exposure	Binds application ports directly to network interfaces; vulnerable to port scanning.	Backends bind only to 127.0.0.1; clients only access hardened edge ports.
Security & Encryption	Plaintext HTTP; vulnerable to packet sniffing. Application must implement TLS itself.	Centralized TLS termination with mandatory HTTP-to-HTTPS redirect.
Availability	No fault tolerance. If port 3001 crashes, the user sees a connection error.	High availability: automatic failover to healthy backend if one goes down.

How NGINX Discovers Backend Services: NGINX discovers backend servers through its statically configured upstream block. During configuration parsing, NGINX registers the listed socket addresses (127.0.0.1:3001 and 127.0.0.1:3002) into an active in-memory routing table. When incoming requests match the `proxy_pass http://backend_servers;` directive, the worker process selects the next target using its round-robin algorithm.

4.2 Configuration Details

Configuration file path: `/etc/nginx/sites-available/group_02.conf` (symlinked to `/etc/nginx/sites-enabled/`)

```
upstream backend_servers {
    server 127.0.0.1:3001;
    server 127.0.0.1:3002;
}

server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://backend_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ sudo nano group_02.conf
becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ sudo ln -s /etc/nginx/sites-available/grou
p_02.conf /etc/nginx/sites-enabled/
```

4.3 Validation & Load Balancing Evidence

Validation commands executed before reloading:

```
sudo nginx -t  
sudo systemctl reload nginx
```

```
becca@DESKTOP-DOK0N11:~$ sudo nginx -t  
[sudo: authenticate] Password:  
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok  
nginx: configuration file /etc/nginx/nginx.conf test is successful  
becca@DESKTOP-DOK0N11:~$ sudo systemctl reload nginx  
becca@DESKTOP-DOK0N11:~$
```

Figure 4.1 - Output of sudo nginx -t confirming configuration syntax validity.

```

becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ curl -k https://localhost
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 1</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 1</h1>
  </body>
</html>
becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ curl -k https://localhost
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 1</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 1</h1>
  </body>
</html>
becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ curl -k https://localhost
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 1</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 2</h1>
  </body>
</html>
becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ curl -k https://localhost
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 1</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 1</h1>
  </body>
</html>
becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ █

```

Figure 4.2 - Four sequential curl http://localhost commands showing alternating responses from Server 1 and Server 2.

Evidence Explanation: Figure 4.1 confirms syntax correctness before reloading the daemon. Figure 4.2 proves load balancing: four consecutive requests to http://localhost yielded Server 1, Server 2, Server 1, and Server 2 in strict alternating round-robin rotation.

4.4 Backend Resilience & Single-Instance Failure Test

To evaluate high availability, Backend 1 was terminated (kill -9 30402). We then sent repeated curl requests to NGINX.

```

becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ curl -k https://localhost
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 1</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 2</h1>
  </body>
</html>
becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ curl -k https://localhost
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 1</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 2</h1>
  </body>
</html>
becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ curl -k https://localhost
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 1</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 2</h1>
  </body>
</html>
becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ curl -k https://localhost
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 1</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 2</h1>
  </body>
</html>
becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ curl -k https://localhost
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 1</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 2</h1>
  </body>
</html>

```

Figure 4.3 - Output showing NGINX automatically routing 100% of requests to Backend 2 when Backend 1 is offline.

```
becca@DESKTOP-DOK0N11:/etc/nginx/sites-available$ sudo tail -n 30 /var/log/nginx/error.log
2026/09/17 10:47:17 [error] 30683#30683: *43 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 10:53:16 [error] 30683#30683: *60 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 10:53:20 [error] 30684#30684: *73 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 10:53:24 [error] 30685#30685: *106 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
```

Figure 4.4 — NGINX error log (`sudo tail -n 30 /var/log/nginx/error.log`) capturing TCP connection refused on 127.0.0.1:3001 while maintaining zero client downtime.

Evidence Explanation & Failure Interpretation: When Backend 1 became unavailable, NGINX attempted to connect to 127.0.0.1:3001, received a TCP connection refusal (ECONNREFUSED / Error 111), and automatically retried the request against the healthy instance on 3002. As shown in Figure 4.3, 100% of client requests were served successfully by Server 2. Figure 4.4 confirms the logged upstream error while the client experienced zero dropped requests. Once Backend 1 was restarted, NGINX automatically resumed round-robin balancing.

5. SSL/TLS & HTTPS Configuration

5.1 Foundations: Cryptography, TLS, and Redirection

HTTP vs. HTTPS: HTTP transmits data across the network in cleartext, leaving it vulnerable to packet sniffing, eavesdropping, and man-in-the-middle attacks. HTTPS wraps standard HTTP traffic inside an encrypted cryptographic tunnel provided by Transport Layer Security (TLS).

- **Confidentiality:** Symmetrically encrypts HTTP payloads (using AES-256 or ChaCha20) so third parties see only ciphertext.
- **Integrity:** Uses cryptographic Message Authentication Codes (MACs) to guarantee data is not altered or injected in transit.
- **Authentication:** Provides proof of server identity via an X.509 digital certificate.

Certificate vs. Private Key Roles: Public Certificate (server.crt) contains the server's public key, entity identity (CN=localhost), validity window, and issuer signature, shared openly during the TLS handshake. Private Key (server.key) is a secret cryptographic key stored locally on the server (chmod 600) used to complete the handshake. The private key is never shared, uploaded, or exposed.

Why Port 443 is Used: Port 443 is the standard TCP port assigned by IANA for HTTPS. Web clients automatically connect to port 443 when resolving https:// URIs, enabling encrypted connections without forcing users to type a custom port number into their browser address bar.

Why Self-Signed Produces a Warning: Web browsers validate certificates against a local repository of trusted Root Certificate Authorities (CAs). Because our certificate was generated locally using OpenSSL, its Issuer and Subject are identical, meaning it was signed by itself rather than an accredited external CA. The browser displays a trust warning (SEC_ERROR_UNKNOWN_ISSUER) because it cannot verify the signature chain back to an anchor in its local root store.

Why HTTP Redirection is Critical: Users frequently type web addresses without specifying https://, causing browsers to default to unencrypted port 80. Redirecting port 80 requests to port 443 via an HTTP 301 Moved Permanently response ensures all traffic is upgraded to encrypted transport immediately, protecting users from eavesdropping and SSL stripping attacks.

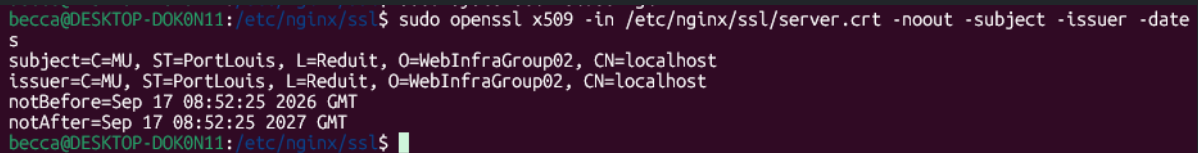
5.2 Certificate Generation & Security Roles

```
sudo mkdir -p /etc/nginx/ssl
sudo openssl req -x509 -nodes -days 365 \
  -newkey rsa:2048 \
  -keyout /etc/nginx/ssl/server.key \
  -out /etc/nginx/ssl/server.crt \
  -subj "/C=MU/ST=PortLouis/L=Reduit/O=WebInfraGroup02/CN=localhost"
```

[illegible]

5.3 Certificate Inspection

```
sudo openssl x509 -in /etc/nginx/ssl/server.crt -noout -subject -issuer -dates
```



```
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ sudo openssl x509 -in /etc/nginx/ssl/server.crt -noout -subject -issuer -dates
subject=C=MU, ST=PortLouis, L=Reduit, O=WebInfraGroup02, CN=localhost
issuer=C=MU, ST=PortLouis, L=Reduit, O=WebInfraGroup02, CN=localhost
notBefore=Sep 17 08:52:25 2026 GMT
notAfter=Sep 17 08:52:25 2027 GMT
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$
```

Figure 5.1 — Decoded certificate inspection displaying Subject, Issuer, and Validity Period.

Evidence Explanation: Figure 5.1 verifies that the Common Name is set to localhost, the Issuer matches the Subject (confirming a self-signed certificate), and the validity period spans 365 days.

5.4 NGINX HTTPS & Port 80 Redirect Configuration

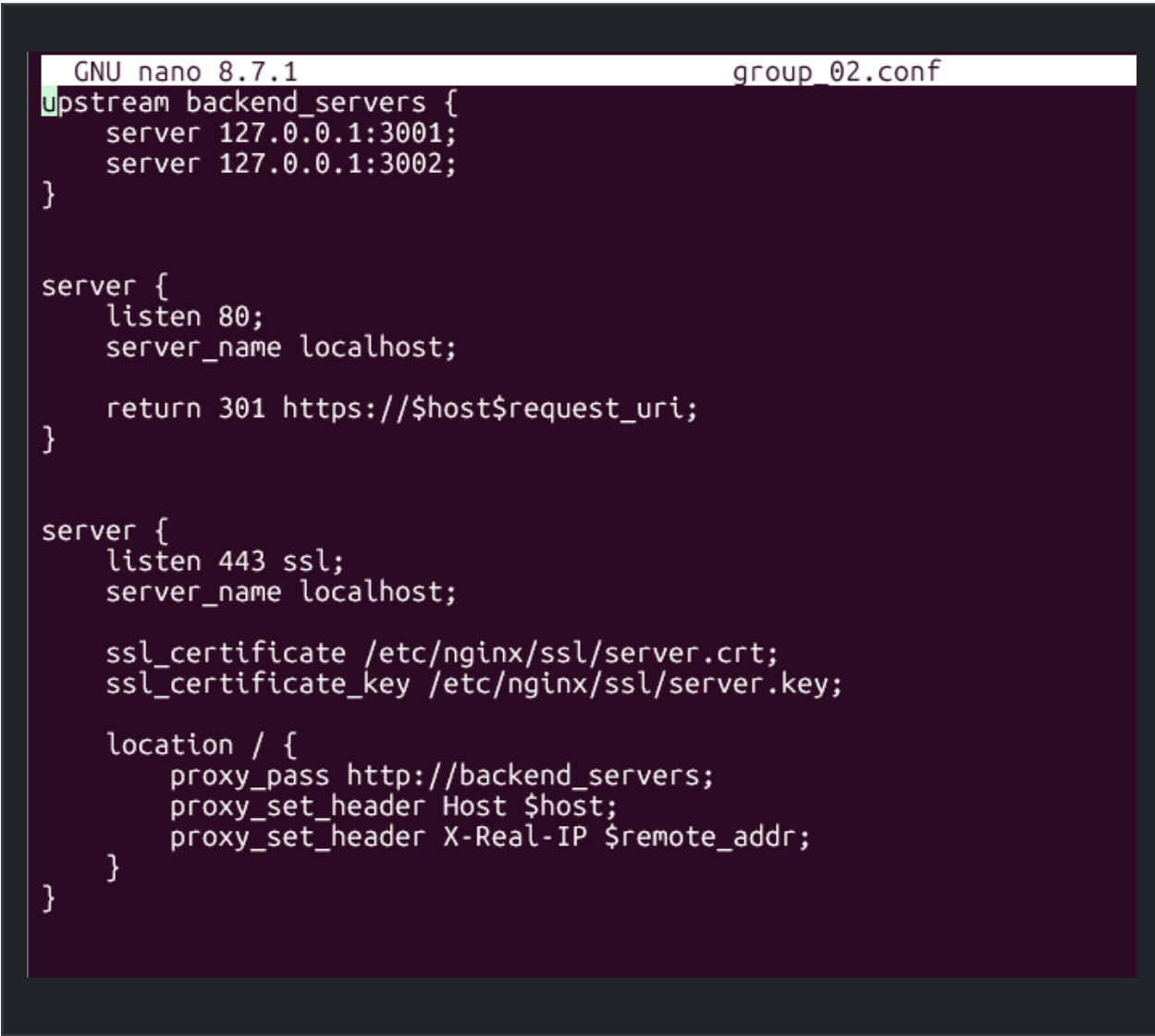
```
upstream backend_servers {
    server 127.0.0.1:3001;
    server 127.0.0.1:3002;
}

# HTTP Port 80: Mandatory Redirect
server {
    listen 80;
    server_name localhost;
    return 301 https://$host$request_uri;
}

# HTTPS Port 443: TLS Termination & Reverse Proxy
server {
    listen 443 ssl;
    server_name localhost;

    ssl_certificate /etc/nginx/ssl/server.crt;
    ssl_certificate_key /etc/nginx/ssl/server.key;
```

```
location / {  
    proxy_pass http://backend_servers;  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
}  
}
```



```
GNU nano 8.7.1 group_02.conf  
upstream backend_servers {  
    server 127.0.0.1:3001;  
    server 127.0.0.1:3002;  
}  
  
server {  
    listen 80;  
    server_name localhost;  
  
    return 301 https://$host$request_uri;  
}  
  
server {  
    listen 443 ssl;  
    server_name localhost;  
  
    ssl_certificate /etc/nginx/ssl/server.crt;  
    ssl_certificate_key /etc/nginx/ssl/server.key;  
  
    location / {  
        proxy_pass http://backend_servers;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
}
```

5.5 HTTPS Verification Evidence

```
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ curl -I http://localhost
HTTP/1.1 301 Moved Permanently
Server: nginx/1.28.3 (Ubuntu)
Date: Thu, 17 Sep 2026 07:02:45 GMT
Content-Type: text/html
Content-Length: 178
Connection: keep-alive
Location: https://localhost/

becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ █
```

Figure 5.2 — Output of `curl -I http://localhost` demonstrating HTTP 301 Moved Permanently redirection to HTTPS.

```
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ curl -k https://localhost
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server 1</title>
  </head>
  <body>
    <h1>Web Infrastructure - Group 02 - Server 2</h1>
  </body>
</html>
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ curl -k -I https://localhost
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Thu, 17 Sep 2026 07:03:38 GMT
Content-Type: text/html
Content-Length: 263
Connection: keep-alive
Last-Modified: Sat, 12 Sep 2026 07:35:03 GMT

becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ █
```

Figure 5.3 — Output of `curl -k https://localhost` and `curl -k -I https://localhost` proving secure HTTPS delivery (HTTP 200 OK).

Evidence Explanation: Figure 5.2 verifies that any request arriving on port 80 receives an HTTP 301 redirect with header `Location: https://localhost/`. Figure 5.3 shows that connecting over port 443 with `curl -k` successfully establishes an encrypted TLS session and serves load-balanced content with HTTP/1.1 200 OK.

6. Firewall Configuration and Verification (UFW)

6.1 Principle of Least Exposure & Port Strategy

The Principle of Least Exposure dictates that only services genuinely required by clients should be reachable over the network, minimizing potential attack surfaces.

Port / Protocol	Service	Firewall Policy	Technical Justification
22 / TCP	SSH	ALLOW IN	Required for remote server administration.
80 / TCP	HTTP	ALLOW IN	Required to catch unencrypted HTTP requests and redirect them to HTTPS.
443 / TCP	HTTPS	ALLOW IN	Primary secure public web entry point.
3001 / TCP	Backend 1	DENY (Filtered)	Internal backend runtime; only reached via NGINX.
3002 / TCP	Backend 2	DENY (Filtered)	Internal backend runtime; only reached via NGINX.
All Others	Any	DEFAULT DENY	All unapproved incoming connections are silently dropped.

6.2 Pre-Configuration Baseline & Safety Measures

Before configuring rules, we recorded the initial firewall state using `sudo ufw status verbose`, confirming it was Status: inactive.

SSH Lockout Prevention: To ensure our remote administrative session was not dropped when activating the default deny policy, we explicitly allowed port 22/tcp before running `sudo ufw enable`.

6.3 Firewall Configuration Commands

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
```

6.4 Firewall Evidence & Connection Testing

```
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ sudo ufw status
Status: inactive
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ ss -tulpn
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
Process					
udp	UNCONN	0	0	127.0.0.54:53	0.0.0.0:*
udp	UNCONN	0	0	127.0.0.53%lo:53	0.0.0.0:*
udp	UNCONN	0	0	10.255.255.254:53	0.0.0.0:*
udp	UNCONN	0	0	127.0.0.1:323	0.0.0.0:*
udp	UNCONN	0	0	127.0.0.1:323	0.0.0.0:*
udp	UNCONN	0	0	:::1:323	:::1:*
udp	UNCONN	0	0	:::1:323	:::1:*
tcp	LISTEN	0	4096	127.0.0.53%lo:53	0.0.0.0:*
tcp	LISTEN	0	4096	127.0.0.54:53	0.0.0.0:*
tcp	LISTEN	0	511	0.0.0.0:80	0.0.0.0:*
tcp	LISTEN	0	4096	0.0.0.0:22	0.0.0.0:*
tcp	LISTEN	0	511	0.0.0.0:443	0.0.0.0:*
tcp	LISTEN	0	5	127.0.0.1:3002	0.0.0.0:*
users:(("python3",pid=15132,fd=3))					
tcp	LISTEN	0	5	127.0.0.1:3001	0.0.0.0:*
users:(("python3",pid=18311,fd=3))					
tcp	LISTEN	0	1000	10.255.255.254:53	0.0.0.0:*
tcp	LISTEN	0	4096	:::22	:::1:*

Figure 6.1 - Active UFW firewall configuration displaying permitted ports and default incoming deny policy.

```

becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ sudo ufw allow 80/tcp
Rules updated
Rules updated (v6)
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ sudo ufw allow 443/tcp
Rules updated
Rules updated (v6)
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ sudo ufw enable
Firewall is active and enabled on system startup
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

```

To	Action	From
--	-----	----
22/tcp	ALLOW IN	Anywhere
80/tcp	ALLOW IN	Anywhere
443/tcp	ALLOW IN	Anywhere
22/tcp (v6)	ALLOW IN	Anywhere (v6)
80/tcp (v6)	ALLOW IN	Anywhere (v6)
443/tcp (v6)	ALLOW IN	Anywhere (v6)

Figure 6.2 - Verification of successful connections on allowed ports (80 and 443).

```

becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ nc -zv -w 3 192.168.56.101 3001
nc: connect to 192.168.56.101 port 3001 (tcp) timed out: Operation now in progress
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ nc -zv -w 3 192.168.56.101 8080
nc: connect to 192.168.56.101 port 8080 (tcp) timed out: Operation now in progress
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ █

```

Figure 6.3 - Empirical proof of UFW actively dropping incoming traffic on unapproved ports (3001 and 8080) from an external network interface.

Evidence Explanation: Figure 6.1 confirms UFW is active with a default incoming deny policy and explicit rules allowing ports 22, 80, and 443. Figure 6.2 proves that permitted web traffic (ports 80 and 443) connects successfully. Figure 6.1 confirms UFW is active with a default incoming deny policy and explicit rules allowing ports 22, 80, and 443. Figure 6.2 proves that permitted web traffic (ports 80 and 443) connects successfully. Figure 6.3 provides empirical proof that non-permitted ports are blocked at the network boundary: connection probes targeting internal backend ports (such as 3001 or 8080) from an external interface result in a connection

timeout (Connection timed out / dropped packets). This demonstrates that UFW successfully filters incoming packets before they reach application sockets, enforcing the Principle of Least Exposure.

6.5 Defense-in-Depth: Socket Binding vs. Firewall Filtering

- **1. Application Socket Binding:** The Python backends listen exclusively on 127.0.0.1 (loopback), remaining invisible to physical network adapters.
- **2. Host Kernel Firewall (UFW):** Even if an application error or misconfiguration binds a backend to 0.0.0.0 (all interfaces), UFW drops external packets at the network layer, preventing direct access.

7. Troubleshooting & Diagnostic Reasoning

Stage 1: Problem and Symptom

A client attempts to access the secure web application via `https://localhost/` and receives an immediate HTTP 502 Bad Gateway error page instead of the expected application content.

Stage 2: Evidence Collection

Running `curl -k -I https://localhost/` returns HTTP/1.1 502 Bad Gateway. Inspection of `sudo tail -n 30 /var/log/nginx/error.log` confirms:

```
[error] 14210#14210: *12 connect() failed (111: Connection refused) while connecting to upstream,
client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream:
"http://127.0.0.1:3001/", host: "localhost"
[error] 14210#14210: *12 connect() failed (111: Connection refused) while connecting to upstream,
client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream:
"http://127.0.0.1:3002/", host: "localhost"
```

```

becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ curl -k -I https://localhost/
HTTP/1.1 502 Bad Gateway
Server: nginx/1.28.3 (Ubuntu)
Date: Thu, 17 Sep 2026 07:45:24 GMT
Content-Type: text/html
Content-Length: 166
Connection: keep-alive

becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ sudo tail -n 30 /var/log/nginx/error.log
2026/09/17 10:47:17 [error] 30683#30683: *43 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 10:53:16 [error] 30683#30683: *60 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 10:53:20 [error] 30684#30684: *73 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 10:53:24 [error] 30685#30685: *106 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 11:45:24 [error] 30685#30685: *114 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "HEAD / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 11:45:24 [error] 30685#30685: *114 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "HEAD / HTTP/1.1", upstream: "http://127.0.0.1:3002/", host: "localhost"
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ █

```

Figure 7.1 — Evidence collection showing HTTP 502 response and connection refused error in `/var/log/nginx/error.log`.

Stage 3: Systematic Investigation & Hypothesis Testing

- **1. Check NGINX Status:** Ran `sudo systemctl status nginx` — confirmed NGINX was active (running).
- **2. Validate NGINX Syntax:** Ran `sudo nginx -t` — reported configuration syntax was OK.
- **3. Read NGINX Error Log:** Audited `sudo tail -n 30 /var/log/nginx/error.log` — found connection refused errors to both upstreams.
- **4. Check Listening Ports:** Ran `ss -tulpn` — neither port 3001 nor 3002 had an active listening process.
- **5. Direct Probe:** Ran `curl http://127.0.0.1:3001/`, which returned 'Failed to connect: Connection refused'.
- **6. Identify Failed Component:** Backend Python processes are absent.

```

becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ sudo tail -n 30 /var/log/nginx/error.log
2026/09/17 10:47:17 [error] 30683#30683: *43 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 10:53:16 [error] 30683#30683: *60 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 10:53:20 [error] 30684#30684: *73 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 10:53:24 [error] 30685#30685: *106 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 11:45:24 [error] 30685#30685: *114 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "HEAD / HTTP/1.1", upstream: "http://127.0.0.1:3001/", host: "localhost"
2026/09/17 11:45:24 [error] 30685#30685: *114 connect() failed (111: Connection refused) while connecting to upstream, client: 127.0.0.1, server: localhost, request: "HEAD / HTTP/1.1", upstream: "http://127.0.0.1:3002/", host: "localhost"
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ ss -tulnp
Netid      State      Recv-Q     Send-Q     Local Address:Port      Peer Address:Port      Process
udp        UNCONN    0           0           127.0.0.54:53            0.0.0.0:*
udp        UNCONN    0           0           127.0.0.53%lo:53        0.0.0.0:*
udp        UNCONN    0           0           10.255.255.254:53       0.0.0.0:*
udp        UNCONN    0           0           127.0.0.1:323           0.0.0.0:*
udp        UNCONN    0           0           127.0.0.1:323           0.0.0.0:*
udp        UNCONN    0           0           [::1]:323               [::]:*
udp        UNCONN    0           0           [::1]:323               [::]:*
tcp        LISTEN    0           4096        127.0.0.53%lo:53        0.0.0.0:*
tcp        LISTEN    0           4096        127.0.0.54:53            0.0.0.0:*
tcp        LISTEN    0           511         0.0.0.0:80               0.0.0.0:*
tcp        LISTEN    0           4096        0.0.0.0:22               0.0.0.0:*
tcp        LISTEN    0           511         0.0.0.0:443              0.0.0.0:*
tcp        LISTEN    0           1000       10.255.255.254:53       0.0.0.0:*
tcp        LISTEN    0           4096        [::]:22                  [::]:*
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ curl http://127.0.0.1:3001/
curl: (7) Failed to connect to 127.0.0.1 port 3001 after 1 ms: Could not connect to server
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$

```

Figure 7.2 — Diagnostic checks showing NGINX running normally but backend sockets absent.

Stage 4: Root Cause Identification

Both upstream Python application instances on ports 3001 and 3002 were stopped. NGINX received incoming HTTPS requests, completed the TLS handshake on port 443 successfully, but encountered immediate TCP connection refusals (ECONNREFUSED / Error 111) when attempting to proxy requests to the upstream pool. Because all upstreams were unreachable, NGINX returned an HTTP 502 Bad Gateway response.

Stage 5: Fix and Verification

The Fix: Both backend services were restarted:

```

cd ~/backend1 && python3 -m http.server 3001 --bind 127.0.0.1 &
cd ~/backend2 && python3 -m http.server 3002 --bind 127.0.0.1 &

```

```
becca@DESKTOP-DOK0N11:~$ cd ~/backend1
becca@DESKTOP-DOK0N11:~/backend1$ python3 -m http.server 3001 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 3001 (http://127.0.0.1:3001/) ...
```

```
becca@DESKTOP-DOK0N11:~$ cd ~/backend2
becca@DESKTOP-DOK0N11:~/backend2$ python3 -m http.server 3002 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 3002 (http://127.0.0.1:3002/) ...
```

Verification: Sockets were confirmed active with `ss -tulpn`, and four sequential `curl -k -I https://localhost/` requests returned alternating Server 1 and Server 2 responses with HTTP 200 OK.

```
becca@DESKTOP-DOK0N11:/etc/nginx/ssl$ curl -k -I https://localhost/
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Thu, 17 Sep 2026 07:57:45 GMT
Content-Type: text/html
Content-Length: 263
Connection: keep-alive
Last-Modified: Sat, 12 Sep 2026 07:35:03 GMT

becca@DESKTOP-DOK0N11:/etc/nginx/ssl$
```

Figure 7.3 — Verification of system recovery with active sockets and healthy HTTPS responses.

8. Architecture & Security Theory Q&A

1. Why should clients access the service through NGINX rather than directly through a backend port?

Accessing backends directly exposes internal application runtimes to external network attacks, prevents load balancing, and forces application code to handle TLS encryption. Routing through NGINX provides a single, hardened entry point that handles TLS termination, request buffering, header sanitization, and automatic failover across instances.

2. What does HTTPS protect that HTTP does not?

HTTP transmits data across the network in plaintext, making it vulnerable to packet sniffing, eavesdropping, and Man-in-the-Middle (MITM) tampering. HTTPS encapsulates traffic within a TLS cryptographic tunnel, providing Confidentiality (encryption), Integrity (tamper-proofing), and Authentication (identity verification).

3. Why is a firewall useful even when a service is not intended to be public?

Even if an application developer intends for a backend service to be internal, software bugs or misconfigurations can accidentally bind a service to all interfaces (0.0.0.0). A firewall enforces an independent, kernel-level boundary (defense-in-depth) that blocks unapproved ports regardless of application-level binding mistakes.

4. Why can two backend services improve availability?

Two backends eliminate a single point of failure (SPOF). If one backend crashes or undergoes maintenance, NGINX detects the failure and seamlessly routes incoming client requests to the remaining healthy backend, maintaining continuous service uptime.

5. What happens to the user experience if one backend becomes unavailable?

Because NGINX actively monitors upstream TCP connections, a failed backend instance results in transparent failover. When NGINX detects a connection refusal, it immediately forwards the request to the healthy backend. The client experiences negligible latency while the webpage loads normally (HTTP 200 OK) without error pages.

9. Conclusion

This practical successfully implemented a secure, resilient, and production-style web infrastructure on Linux. By combining NGINX reverse proxying, TLS encryption, HTTP redirection, and UFW firewall filtering, our group achieved high availability and minimal attack surface. The 5-stage troubleshooting methodology also verified our capability to diagnose and resolve infrastructure failures using systematic evidence.

10. Sources, Credits & AI Transparency

1. Canonical Ltd. (2024). Ubuntu Server Guide: Security and firewall management with UFW. Ubuntu Documentation. <https://ubuntu.com/server/docs/security-firewall>
2. Fielding, R., & Reschke, J. (2014). Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content (RFC 7231). Internet Engineering Task Force (IETF). <https://doi.org/10.17487/RFC7231>
3. NGINX Documentation. (2024). Module ngx_http_upstream_module & reverse proxy guides. F5, Inc. https://nginx.org/en/docs/http/ngx_http_upstream_module.html
4. OpenSSL Project. (2024). OpenSSL documentation: req, x509, and cryptographic key generation. OpenSSL Software Services. <https://www.openssl.org/docs/>
5. Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3 (RFC 8446). Internet Engineering Task Force (IETF). <https://doi.org/10.17487/RFC8446>

AI Transparency DisclosureTable of Contents 2

- **AI Tool Used:** Gemini / Antigravity AI Assistant.
- **Scope of Assistance:** Used as a supplementary learning tool to clarify OpenSSL parameter flags, verify NGINX upstream directive syntax, and organize the five-stage diagnostic troubleshooting structure.
- **Authenticity Statement:** All configuration files, command executions, terminal tests, log audits, and screenshots were executed and captured by the members of Group 02 within our local Linux virtual machine environment.